Drawing: A New Way To Search

Doodle classification: reproduction study and XGBoost extension

ARJUN SRIKANTH - PES1UG24AM920 | NIKHIL JOSHI - PES1UG24AM441

Machine Learning mini-project | 22 October 2026

Problem statement

Given a simple hand-drawn image, predict its category from a fixed vocabulary. The project compares the reference paper's classical and neural model families on 3, 10 and 50 categories and adds XGBoost. Drawing-based search is the motivation; the implemented output is a category label and model scores, without photograph retrieval.

Dataset and experimental split

Google Quick, Draw! bitmap drawings are 28 x 28 grayscale images represented by 784 pixel values in [0,255]. The code selects the first 20,000 examples per class and uses seed 42 to split each class into 16,000 training, 2,000 validation and 2,000 test examples. Models for a class set reuse the same saved split. The three-class vocabulary is eraser, belt and banana; the larger vocabularies follow the reference source.

Classes	Training	Validation	Test
3	48,000	6,000	6,000
10	160,000	20,000	20,000
50	800,000	100,000	100,000

Approach

Logistic regression: multinomial linear baseline on raw pixels, L-BFGS and 100 iterations. **SVM:** linear, RBF, polynomial and sigmoid kernels on pixels scaled to [-1,1], with a 500-iteration cap; evaluated on 3 and 10 classes. These caps can stop before convergence.

CNNs: V1 uses two convolution/pooling stages with 14 filters of size 3 x 3, followed by Dense(128) and a class softmax. V2 removes the second stage, V3 removes pooling from V2 and V4 reduces dense width to 64. Eighteen raw/binarized configurations use Adam, batch 500 and 100 epochs. Raw CNN inputs retain [0,255].

Transfer learning: VGG19, MobileNet, ResNet50 and InceptionV3 are evaluated on three classes. Images are resized and repeated to three channels; frozen features are cached before training a new head. Inception additionally fine-tunes upper layers. **XGBoost:** boosted trees use the same 784 raw pixels. Optuna tunes seven settings using validation accuracy, while early stopping monitors validation log-loss with patience 50.

Implementation

The source modules provide data preparation, model training and shared recording. The pipeline notebook explains the training sequence; the demo notebook trains a small baseline and loads saved CNN/XGBoost models for prediction. Keras callbacks save training checkpoints; XGBoost writes model files. CSV/JSON records, learning curves and confusion matrices preserve experiment evidence. Extra SVM data-size checks and XGBoost continuation runs remain separate recorded experiments.

Results and working demonstration

All entries below are validation accuracy (%), best tested configuration within each family. A dash means the combination was not run. Precise results and settings are retained in the experiment ledger.

Family	3 classes	10 classes	50 classes
Logistic regression	79.27	65.79	43.31
SVM	84.00	74.62	-
CNN	91.72	88.77	81.21
Transfer learning	95.35	-	-
Final XGBoost	91.65	89.84	77.95

The ledger contains 46 configurations and follow-up runs, including 18 CNN runs and eight SVM data-size checks. Among the tested 50-class configurations, CNN V1 on raw input gives the strongest validation result. Its independently checked Mac test accuracy is 81.071% (81.1% in the demo). Final XGBoost test accuracies are 90.80%, 89.79% and 77.886% for 3, 10 and 50 classes. Extended training improves 50-class test accuracy from 77.44% to 77.89%.

Historical selected fit times are 431.43 seconds for the 50-class CNN V1 and 7,191.10 seconds for final XGBoost, a ratio of 16.67 on the recorded hardware. This excludes total tuning cost and is not a hardware-independent speed claim. Historical training used an RTX 4070 Laptop GPU / Intel i7-14650HX system; Mac demonstration uses CPU inference.

Working code and inference

The verified demo displays labelled examples, fits three-class logistic regression, evaluates the saved CNN on 100,000 test images and compares CNN/XGBoost on a supplied drawing. A fresh Mac kernel executed all ten code cells successfully. The live canvas converts strokes to grayscale, inverts a light background, crops, pads and resizes to 28 x 28. Both models receive the same processed pixels. Real endpoint checks passed blank input and star/umbrella examples with optional stroke adaptation on and off.

Interpretation and limitations

A version-selection overwrite is observable in the original CNN source, but the exact architecture used for every historical published row remains unknown. Larger-data SVM reruns support data-size/iteration-cap effects. Frozen BatchNorm behavior is a plausible transfer-learning explanation rather than a controlled proof. CNN V3 differences remain unresolved. The modern implementation is a documented reproduction study, not an assertion of identical historical execution.

Most runs use one seed, tuning effort differs by model and some timings involve different devices or concurrent jobs. Row splits can contain rare identical bitmaps; one 3-class train/test overlap was found. Dataset accuracy does not establish accuracy on new human drawings. Optional stroke adaptation helps tested vector-style inputs but can harm bitmap roundtrips. Unsupported categories still receive a supported label; scores are not calibrated guarantees.

Conclusion

A compact raw-input CNN provides the best measured 50-class validation accuracy among the tested configurations. XGBoost is a meaningful additional family but is sensitive to pixel appearance. Future work should evaluate new human drawings, repeated seeds, stricter deduplication and feature-based XGBoost. The existing trained models support the live demonstration without repeating the expensive historical runs.

Sources and provenance

[1] Phu, Xiao and Muindi. *Drawing: A New Way To Search*, Stanford CS229, 2018. cs229.stanford.edu/proj2018/report/33.pdf
[2] Google Creative Lab. Quick, Draw! dataset. github.com/googlecreativelab/quickdraw-dataset
[3] Supplied source and historical experiments: project implementation and experiment records, with supplied teammate data/model bundles.
[4] Evidence: results/all_results.csv and verification/presentation_*_20261022.json. Supplied experiments and later Mac verification/canvas work are distinguished. State individual contributions according to actual work performed.